ECO-VIBE: APPLICATION FOR BUYING AND SELLING SECONDHAND CLOTHES

Carol Stefanya Velasco Rodríguez, David Eduardo Muñoz Mariño
Universidad Distrital Francisco Jose de Caldas
Faculty of Engineering - Systems Engineering
Bogotá D.C., February 2025

CONTENTS

LIST OF FIGURES

ECO-VIBE: APPLICATION FOR BUYING AND SELLING SECONDHAND CLOTHES

***Abstract*—ECO-VIBE is an e-commerce platform developed to support the buying and selling of secondhand clothing. The system integrates Java and Python web services to manage product listings, search functions, and user interactions. Its design focuses on efficiency, ensuring smooth operations across different components. This report details the platform's architecture, backend development, and key processes, explaining how various technologies work together to enhance functionality and scalability.**

I. Introduction

The rise of digital platforms has changed how people buy and sell secondhand clothing. ECO-VIBE aims to create an easy-to-use and efficient online marketplace where users can quickly buy and sell clothes.

This project focuses not only on user experience but also on integrating web services developed in Java and Python to manage the platform's different features. This architecture improves product listings, searching, and purchasing.

This report explains the development, design, and implementation of the application, covering both business logic and the connection between backend services.

II. User Stories

A survey was carried out on 20 potential users with the aim of knowing their preferences and needs when using a second-hand clothing sales application. The questions aimed to identify the main goals of the users, the desired features in the application, the way the products are displayed, the necessary information about the products and the importance of direct contact with other users.

A. Questions

The questions and their results were:

1. What is your main interest when using a virtual second-hand clothing application?

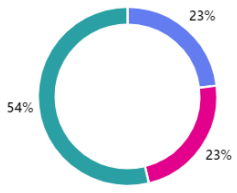

Fig. 1. User interest in second-hand clothing applications

Options:
- Buy second-hand clothes (Blue)
- Sell second-hand clothes (Pink)
- Both options (Green)

2. How would you like the products to be displayed in the app?

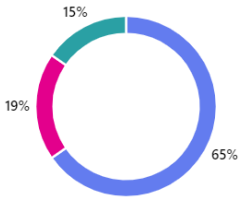

Fig. 2. How users prefer to view products in the app

Options:
- With photos or videos (Blue)
- With opinions and reviews from other users (Pink)
- Others: (Green)
- Both
- All of the above

3. What information about the product would you like to see provided in the app?

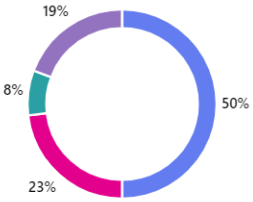

Fig. 3. User preferences for product details

Options:
- Detailed product description (Blue)
- Product condition (new, used, etc) (Pink)
- Size and measurements (Green)
- Others: (Purple)
- All of the above

B. User stories

Once the responses are collected, user stories are created:

- "As a seller, I want to be able to add detailed descriptions and high-quality photographs to show my products in the best way possible"

• ”As a user I want to be able to see the products with photos and detailed descriptions to have a clear idea of what I am buying”

• ”As a buyer, I want to be able to see detailed information about the product, such as condition and measurements, to make an informed decision”

After analyzing user responses, it was observed that most users value the ability to view products with photographs and detailed descriptions. The importance of detailed product information, such as conditions and measurements, was also highlighted. The proposed functionalities will cover the needs of users, and additional suggestions will be taken into account to improve the user experience in the second-hand clothing sales application.

III. Functional and Non-Functional Requirements

A. Functional Requirements

• User Registration: Users create an account by providing a username and password.
• Product Listing: Sellers upload images, descriptions, and prices of their secondhand clothing items.
• Product Search: Buyers search for products using keywords.
• Product Purchase: To buy a product, users enter their address, bank details, and phone number.
• Reviews and Ratings: Buyers leave a review of both the product and the seller after completing a purchase.

B. Non-Functional Requirements

• Usability: The platform is intuitive and easy to navigate.
• Visual Design: The interface has pleasant colors and a clear layout.
• Maintainability: The system allows future updates without affecting functionality.

IV. UML Diagrams

A. Deployment diagram

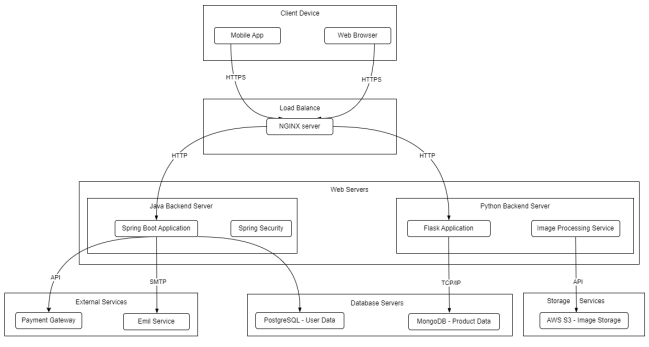

Fig. 4. Deployment diagram

B. Sequence diagram

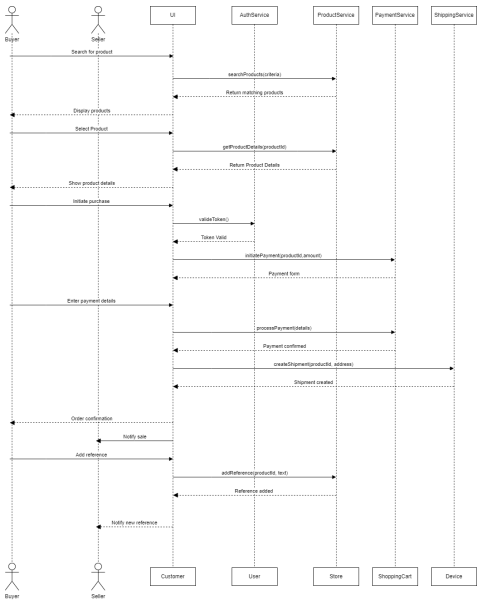

Fig. 5. Sequence diagram

C. Flow diagram

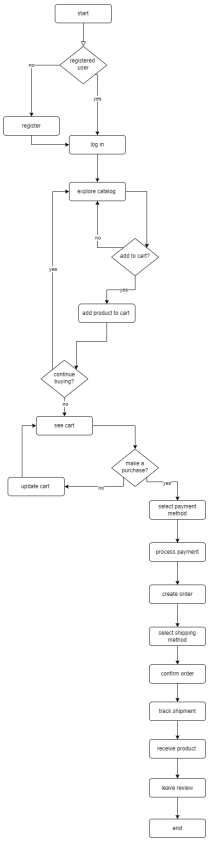

Fig. 6. Flow diagram

D. Activity diagram

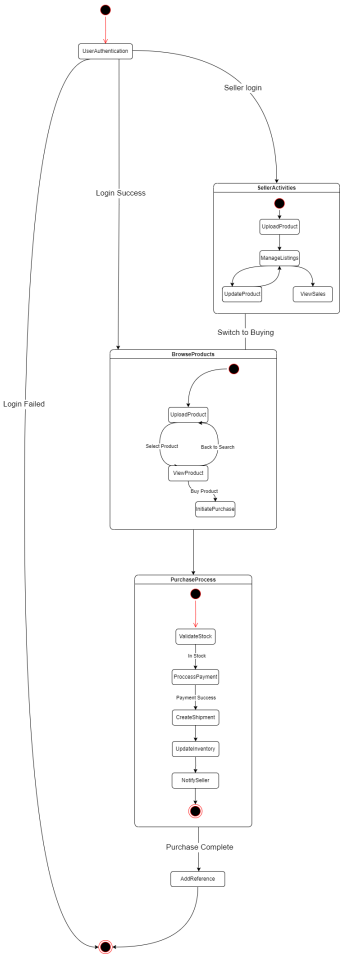

Fig. 7. Activity diagram

V. Conceptual Design

A. Web GUI Mockups

The design of EcoVibe is based on earth tones and brown colors to create a natural and eco-friendly feel, matching the idea of giving clothes a second life. A minimalist style was chosen to keep the platform simple, with a clean and organized look.

Each mockup is designed to make navigation clear and easy, so users don't feel lost. The elements are arranged in a way that helps users complete actions like searching for products, signing up, or making a purchase. There is also an option to rate the seller's service, allowing users to share their experience and help improve the platform. The font style and button placement were carefully selected to enhance readability and accessibility. Additionally, spacing and contrast ensure that the interface remains visually appealing while being functional for all users.

The Mockups are:

- Mockup - Login and Sign Up

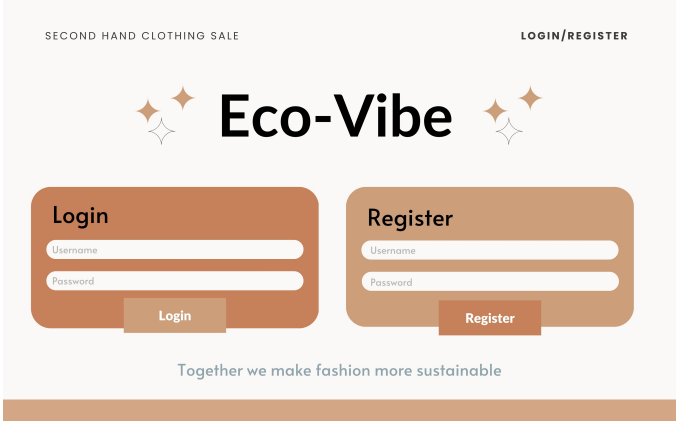

Fig. 8. Login and Sign Up

- Mockup - Upload Product

Fig. 9. Upload Product

- Mockup - Search Product

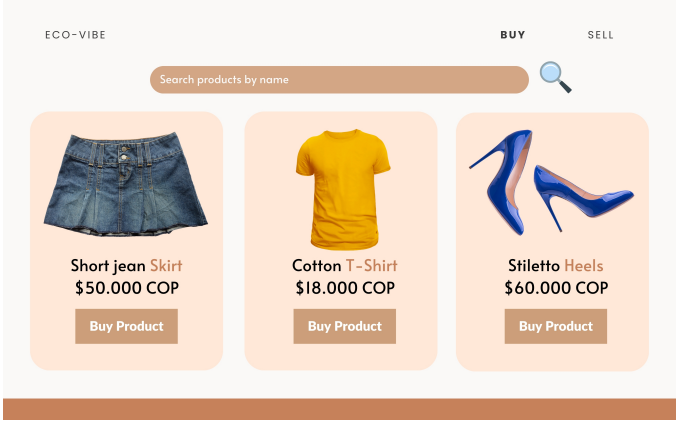

Fig. 10. Search Product

• Mockup - Buy Product

Fig. 11. Buy Product

• Mockup - Rate Product

Fig. 12. Rate Product

B. CRC cards

The CRC cards show the structure and organization of the main components of the EcoVibe application, detailing their responsibilities and interactions. They identify key elements such as users, products, orders, and payments, along with their functions in the system. This representation helps to understand how the different parts of the application work together, making development and maintenance easier.

User	Product	order
- Register in the system - Login/Logout - Update profile information - View order history - Create product listings - Leave reviews - Manage shopping cart	- Store product information - Update product details - Display product information - Manage product status - Track inventory - Show product images	- Create new orders - Calculate total cost - Update order status - Track shipping information - Store order details - Process refunds - Manage order cancellation
- Order - Product - Cart - Review	- User (Seller) - Catalog - Order - Cart - Review	- User - Product - PaymentSystem - ShippingSystem

Cart	Catalog	PaymentSystem
- Add products - Remove products - Calculate total - Update quantities - Clear cart - Process checkout - Save cart items	- Display all products - Filter products - Search products - Sort products - Manage categories - Update product listings	- Process payments - Verify transactions - Handle refunds - Generate receipts - Store payment methods - Validate payment information
- User - Product - Order	- Product - User	- Order - User

ShippingSystem	Review	Message
- Calculate shipping costs - Track shipments - Manage shipping providers - Update delivery status - Generate shipping labels - Handle shipping issues	- Create reviews - Edit reviews - Delete reviews - Display reviews - Calculate ratings - Manage review moderation	- Send messages between user - Store message history - Notify users of new messages - Mark messages as read/unrea - Filter conversations - Archive messages
- Order - User	- User - Product	- User (Sender) - User (Receiver)

Fig. 13. CRC cards

VI. System Architecture

A. Class diagram

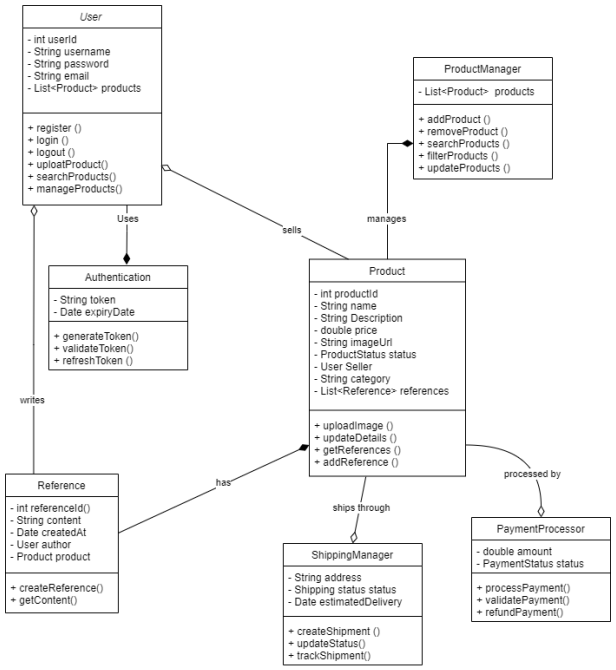

Fig. 14. Class diagram

VII. Backend Definition

A. Java Backend (Spring Boot)

• Technologies and Libraries Used:

Spring Boot is the main framework for backend development. Spring Security implements authentication and authorization using JWT, while Spring Web facilitates the creation of REST APIs.

• Web Services and Endpoints:

Authentication (AuthController.java)

The authentication service includes the endpoint POST /api/auth/login, which receives username and password, returning a JWT token if the credentials are correct. It also has GET /api/auth/user/me, a protected endpoint for verifying authentication.

User Registration (RegistrationController.java)

The user registration service provides POST /api/auth/register, which receives username and password, allowing the creation of a new user.

B. Python Backend (Flask)

• Technologies and Libraries Used:

The Python backend uses Flask as a lightweight framework for creating *REST APIs. Flask-CORS* allows requests from the *frontend*, and *Werkzeug* handles file uploads.

• Web Services and Endpoints:

a) General:

: • *GET /* Main page.

b) File Management:

: • *GET /uploads/filename* Retrieves uploaded files.

c) Product Management:

: • *GET /products* Lists all products.

• *POST /products* Creates a new product.

• *GET /products/int:productid* Retrieves details of a specific product.

d) Product References:

: • *GET /products/int:productid/references* Lists references for a product.

• *POST /products/int:productid/references* Adds a new reference to a product.

C. Connection Between Java Backend and Python Backend

The Spring Boot backend handles authentication, and Flask may require JWT authentication for certain endpoints. Interaction between both can be achieved by validating tokens generated by Spring Boot in Flask to ensure secure requests.

VIII. SOLID Principle

A. Single Responsibility Principle (SRP)

In *Java (Spring Boot)*, each class has a single function: *AuthController.java* handles authentication, *RegistrationController.java* manages user registration, and *UserService.java* encapsulates user logic. In *Python (Flask)*, most functions are in *app.py*, making it harder to separate responsibilities. It is recommended to split them into modules.

B. Open/Closed Principle (OCP)

In *Java*, services like *AuthService.java* can be extended without modifying their core code.

In *Python*, *app.py* could be better structured to allow extension without directly altering the existing code.

C. Liskov Substitution Principle (LSP)

In *Java*, interfaces like *UserDetailsService* enable swapping implementations without affecting the system.

In *Python*, using base classes and inheritance is recommended to improve flexibility.

D. Interface Segregation Principle (ISP)

In *Java*, controllers expose only the necessary methods.

In *Python*, all routes are in *app.py*, so splitting them into specific modules would improve this principle.

E. Dependency Inversion Principle (DIP)

In *Java*, dependency injection *(@Service, @Autowired)* decouples components.

In *Python*, using Blueprints or Factory would help manage dependencies more efficiently.

IX. Conclusions

• *Structure and Organization:* The combination of Java (Spring Boot) and Python (Flask) creates a functional backend, but Flask should be more modular to improve maintenance and scalability.

• *Application of SOLID:* Java follows SOLID principles with a well-defined architecture. Python is functional but needs better separation of responsibilities and dependency management.

• *Importance of User Stories:* Defining user stories was key to understanding client needs and designing features that meet expectations, ensuring a user-focused product.

• *Use of Diagrams:* Diagrams such as architecture, data flow, and use case diagrams helped visualize the system, improve development, and enhance team communication.

• *Mockup Design:* Creating mockups helped define a user-friendly interface before development, improving user experience and reducing later modifications.

• *Scalability and Future Improvements:* It is recommended to refine modularity, improve system documentation, and optimize backend integration for future expansions.